ВСЕРОССИЙСКИЙ КОНКУРС НАУЧНО-ТЕХНОЛОГИЧЕСКИХ ПРОЕКТОВ
«БОЛЬШИЕ ВЫЗОВЫ»

Направление: Большие данные, искусственный интеллект, автоматизированные системы и информационная безопасность

«Agent AI»

Разработка нейросетевой программной среды с динамической самомодификацией

Выполнил: Ролдугин Илья Владимирович
Обучающийся 11 класса
ГБПОУ ВО "ГПК"

2025-2026 уч. год

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 4

1.1. Актуальность и проблема 4

1.2. Постановка задач, цели и целевая аудитория 4

1.3. План реализации 5

1.4. Используемые ресурсы 5

2. ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ И ИНСТРУМЕНТАРИЙ 7

2.1. Динамическое расширение и исполнение кода 7

2.2. Интерактивная визуализация (HTML-вставки) 7

2.3. Долгосрочная персонализация (user_profile) 7

2.4. Автоматизация операционной системы и веб-среды 8

2.5. Система пресетов и многоуровневое управление 8

3. ПОВЫШЕНИЕ КАЧЕСТВА КОДИНГА МОДЕЛИ 9

3.1. Проблема 9

3.2. Режим “Архитектор” 9

3.3. Режим “Проектировщик” 10

3.4. Тестирование и итоги 11

4. ВЗАИМОДЕЙСТВИЕ С GUI 14

4.1. Проблема 14

4.2. Анализ решений 14

4.3. Browser Use 14

4.4. Computer Use 15

5. РЕЗУЛЬТАТЫ И ПЕРСПЕКТИВЫ РАЗВИТИЯ 18

5.1. Практическая значимость проекта 18

5.2. Сравнительный анализ существующих решений 18

5.3. Перспективы развития и будущие цели 18

ЗАКЛЮЧЕНИЕ 20

СПИСОК ЛИТЕРАТУРЫ И ЭЛЕКТРОННЫХ РЕСУРСОВ 21

ПРИЛОЖЕНИЯ 22

Приложение 1. Ссылки на репозиторий и демонстрации 22

Приложение 2. Сравнительная таблица с аналогами 24

Приложение 3. Архитектура, техническая реализация, использованные технологии и ресурсы 27

Приложение 4. Методика и ход тестирования 28
Приложение 5. Скриншоты 32

1. ВВЕДЕНИЕ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Актуальность и проблема

В индустрии разработки ПО на базе ИИ наблюдается переход от простых чат-ботов к автономным агентным системам.

Проблема: большинство существующих клиентов нейросетей обладают статичной архитектурой: их набор инструментов и логика работы жестко заданы разработчиком.

Проект «Agent AI» предлагает принципиально иной подход — создание среды, где ИИ-агент обладает полномочиями по изменению собственного исходного кода в процессе выполнения. Это позволяет системе адаптироваться к любой задаче, дописывая необходимые методы и интегрируя новые инструменты "на лету".

1.2. Постановка задач, цели и целевая аудитория

Общая цель: Создание высокопроизводительного клиента нейросети с архитектурой динамической самомодификации для решения широкого спектра прикладных задач, и с поддержкой расширения через плагины.

Задачи:

- Разработка механизма Runtime-модификации кода и системы исполнения произвольного кода, сгенерированного нейросетью. (Показатель: Успешное выполнение агентом скриптов, меняющих его собственное поведение без перезагрузки ядра программы).
- Разработка адаптивного интерфейса с поддержкой пресетов и визуализации через HTML-вставки. (Показатель: модель может использовать и активно использует в сообщениях графические элементы, созданные с помощью html)
- Разработка механизма для взаимодействия агента с GUI. (Показатель: Успешное автономное прохождение базовых веб-сценариев с минимальным вмешательством пользователя.)
- Разработка системы для обеспечения качественного кодинга модели без вмешательства пользователя в процессе. (Показатель: Значительное повышение качества кодинга модели при том же промпте пользователя)

Целевая аудитория: IT-специалисты и опытные пользователи компьютера (требуется базовое понимание системного администрирования (настройка WSL, Python-окружения, скачивание репозитория с GitHub) для максимального раскрытия потенциала агента).

1.3. План реализации

План (основан на ходе работ, может меняться):

1) Реализация базового клиента LLM для предоставления нейросети возможности взаимодействия с интерпретатором python (Ноябрь 2025).
2) Реализация базового функционала (работа с консолью, поиск в интернете, сохранение персональных данных) (Ноябрь 2025).
3) Разработка web ui и системы сохранения чатов (Декабрь 2025 – Январь 2026)
4) Начало разработки плагина Computer Use (Январь 2026)
5) Начало разработки плагина Browser Use (Февраль 2026)
6) Разработка системы пресетов (Февраль 2026)
7) Работа над улучшением качества кодинга модели (подробнее в главе 3) (Февраль – Март – Апрель 2026)
8) Доработка плагинов для взаимодействия с GUI (подробнее в главе 4) (Апрель 2026)
9) Доработка проекта и создание документации (Май 2026, в процессе)
10) Создание системы работы с большим объёмом информации (RAG)
11) Исправление багов и оптимизация промптов

1.4. Используемые ресурсы

Для реализации проекта использовались свободно распространяемые программные решения (Open Source) (кроме LLM).

Проект разрабатывался и тестировался на ПК (CPU: Intel Core i5-12450H, GPU: встроена в процессор, ОЗУ: 16 Гб), проект не перегружает компьютер, ощутимые зависания не возникают.

Доступ к моделям семейства Gemini осуществлялся через официальные сервера

по API (LLM вычислялся на них). В начале разработки использовались бесплатные квоты (free tier), затем был произведён переход на платные (paid tier 1), которые полностью оплачивались из стартового баланса (промо-кредитов), предоставляемого платформой новым разработчикам.

Подробная архитектура расписана в приложении 3.

2. ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ И ИНСТРУМЕНТАРИЙ

Проект «Agent AI» предоставляет широкий спектр инструментов для автоматизации интеллектуальной и технической деятельности. Отличительной чертой является не только наличие инструментов, но и способ их взаимодействия с пользователем и программной средой.

2.1. Динамическое расширение и исполнение кода

Центральной «фишкой» системы является возможность исполнения произвольного кода. Инструмент python позволяет агенту:

- Решать сложные математические и алгоритмические задачи.
- Создавать вспомогательные скрипты для обработки данных.
- Модифицировать собственные методы в реальном времени, адаптируя ядро системы под конкретные нужды пользователя без перезагрузки приложения.
- Взаимодействовать с компьютером пользователя (читать / менять файлы, выполнять действия через консоль и api (например печать на принтере))

2.2. Интерактивная визуализация (HTML-вставки)

Агент активно использует механизм HTML-вставок (экранирование html-кода (с полной поддержкой java script)). Это позволяет уйти от сухого текстового вывода и предоставлять информацию в виде:

- Динамических панелей управления и кнопок действий.
- Инфографики и визуализации структур данных (например, деревьев файлов или графиков).
- Интерактивных форм обратной связи, позволяющих пользователю управлять ходом выполнения задачи.

Все вставки рендерятся на лету через встроенный Markdown-процессор с поддержкой HTML-вставок (со всроенным стилем Tailwind CSS) и KaTeX для математических формул.

2.3. Долгосрочная персонализация (user_profile)

Инструмент “user_profile” обеспечивает агента «долгосрочной памятью». В отличие от обычного контекста чата, данные в профиле сохраняются между сессиями. Агент автоматически выделяет и запоминает:

• Личные предпочтения пользователя и стиль общения.

• Постоянные инструкции по выполнению задач (например, правила коммитов или форматирования документов).

• Профессиональный контекст (текущие проекты, используемые библиотеки и технологии).

2.4. Автоматизация операционной системы и веб-среды

Благодаря модульной системе плагинов агент обладает инструментами для глубокой автоматизации:

• Computer Use: позволяет ИИ взаимодействовать с интерфейсом операционной системы (движение мыши, нажатие клавиш, анализ скриншотов).

• Browser Use: обеспечивает автономную навигацию по веб-сайтам, поиск информации и взаимодействие с простыми веб-приложениями через специализированное браузерное расширение.

2.5. Система пресетов и многоуровневое управление

Для эффективного взаимодействия реализована система пресетов, которая мгновенно перенастраивает права доступа (ACL), наборы инструментов и личность агента, примеры пресетов:

- “Безопасный”: Максимальная защита системы. Агенту запрещено изменять любые файлы проекта. Он может писать только в папки temp/ и sandbox/
- “Разработчик”: С помощью специальных режимов можно включать в системный промпт исходный код файлов (по разделам: computer_use, browser_use, backend, frontend), чтобы модель знала содержимое файлов, с которыми работает, а не пыталась угадать.
- “Ревьюер”: ограничение на редактирование файлов и специальный промпт для анализа качества кода и поиска багов.
- Более сложный пример: режим Проектировщик (подробнее в главе 3.3)

3. ПОВЫШЕНИЕ КАЧЕСТВА КОДИНГА МОДЕЛИ

3.1. Проблема

ИИ может качественно решить простую задачу кодинга, но при попытке решить сложную (которую можно разбить на много подзадач) могут появиться проблемы:

1) В коде могут появляться заглушки (например, кнопка, которая ничего не делает)
2) Некоторые подзадачи могут быть реализованы значительно хуже, чем таже нейросеть могла бы реализовать, если попросить выполнить только эту подзадачу.
3) Из-за мелкой ошибки в реализации 1 подзадачи, может сломаться всё
4) Модель может галлюцинировать, допуская из-за этого ошибки
5) Контекстное окно модели может забиться, сильно ухудшив качество
6) Агент может сломать то, что работало до его вмешательства (например, полностью переписать файл, но вместо функций, которые уже были в файле написать заглушки), при доработке крупного проекта это самая критичная проблема, так как вместо пользы, ИИ может нанести вред.

3.2. Режим “Архитектор”

Суть подхода: раздробить сложную задачу на подзадачи, распределить подзадачи по разным чатам и добавить багфикс и контроль качества.

Процесс контролируется Чатом “Архитектор” и проходит через следующие этапы:

1. Пользователь обсуждает задачу с Архитектором.
2. Архитектор разбивает задачу на подзадачи, а подзадачи на шаги, формулирует ТЗ для каждой подзадачи
3. Чат-исполнитель выполняет ТЗ подзадачи по 1 шагу за 1 сообщение
4. Затем проходит “Цикл качества”, он состоит из 3 последовательных проверок, если реализация не прошла хотя бы 1, чат-исполнитель дорабатывает решение, и цикл начинается заново, проверки:
   4.1. Валидация: проверка на заглушки и галлюцинации
   4.2. Ревью: проверка на стиль кода и ошибки
   4.3. Тестирование: поиск ошибок путём тестирования

Так как чат Архитектор часто пропускает этапы и отклоняется от инструкций, было принято решение заскриптовать подход (за создание чатов и управление ими отвечает скрипт).

Описанный выше подход исправляет все проблемы, однако создаёт новые:

- Чаты для проверок могут галлюцинировать, выдумывая ошибки, которых нет (из-за этого тратятся лишние токены, и это может ухудшить качество).
- Чаты для проверок не всегда видят полный контекст.
- Чат-исполнитель не всегда видит полный контекст, из-за чего может сделать не то, что нужно.
- Тестирование может зависнуть.
- Значительно увеличивает потребление не кешированных входных токенов (так как создаются разные чаты, которым нужно передавать контекст)

3.3. Режим “Проектировщик”

Из-за низкой эффективности прошлого подхода было принято решение создать новый. Основные проблемы режима “Архитектор” возникали из-за сложности коммуникации между чатами, поэтому в этом подходе все действия происходят в пределах 1 чата.

Сначала Проектировщик, как и Архитектор обсуждает с пользователем задачу, затем составляет план (план делится на блоки, а блоки на шаги), затем (после нажатия пользователем кнопки “старт”) у пользователя появляются кнопки действий (нажатие на которые генерирует промпт, описывающий текущий этап и действие):

1. Далее (перейти к следующему шагу и выполнить его)
2. Поиск багов (просит агента найти баги, но не исправлять
3. Фикс багов
4. Анализ качества кода
5. Рефакторинг

Также есть вспомогательные кнопки:

1. До конца блока: выполняет “Далее”, пока не достигнет конца блока

2. Авто: выполняет “До конца блока”, “ Поиск багов” и “Фикс багов” (если баги нашлись) для каждого блока, в конце ищет и исправляет баги, затем проводит рефакторинг

Недостатки подхода:

1. Контекстное окно только сильнее нагружается
2. Иногда пропускает баги

3.4. Тестирование и итоги

В ходе разработки были проведены многочисленные тесты (на тестовых задачах, таких как написание браузера на qt) для проверки эффективности и отладки скриптов, далее описание финального теста режима “Проектировщик”

Задача: добавить в web ui возможности персонализации

Промпт:

“

замени кнопку "Динамический фон" на "параметры"

сделай меню параметры, в котором будет фон (обычный(тёмный) и динамические (тот, который сейчас + придумай ещё красивые анимированные варианты)), блюр (xl, 2xl, sm, md...), затемнение (проценты), они влияют на все элементы интерфейса

меню будет всплывающим, по центру, фон за ним будет блюрится

пусть фон сообщений чата блюрится

”

Модель: Gemini 3.1 pro

Версия проекта (коммит): 1134617

Результат запуска в обычном режиме:

Задача реализована очень плохо, появились проблемы, не относящиеся к задаче, некоторые проблемы (которые больше всего бросаются в глаза):

1. Фон сообщений в чатах не блюрится и не затемняется
2. Вернулись проблемы с эффектом blur, которые ранее были исправлены (когда в html для тега указывается класс backdrop-blur-* (или blur через фильтр), и он же присутствует в предках (по DOM), размытый фон накладывается на

прозрачный не размытый, этот баг присутствует в браузерах на основе chromium (возможно и в других, не проверял))

3. Старый динамический фон был сильно упрощён и испорчен (хотя, в промпте не было сказано его менять)
4. Оба новых фона очень плохого качества (1 из них из-за бага отображается как чёрный экран, хотя должен быть анимированный фон)

В итоге многие функции (которые были в ТЗ) придётся переделывать, а также исправлять новые баги, возникшие баги аннулируют работу нейросети, так как проще откатить изменения и сделать работу с нуля, чем отлаживать новые баги.

Для запуска в режиме “Проектировщик” в конец промпта допишем инструкции по плану:

“

подготовь план суммарное количество шагов: ~10

3 блока

на каждый фон выдели отдельный пункт плана

”

Далее нажимаем кнопку “старт”, затем “авто”, далее скрипт всё делает автоматически.

Результат:

Задача реализована хорошо, но с мелкими багами, новые фоны реализованы качественно, некоторые проблемы:

1. Фон сообщений в чатах не блюрится и не затемняется
2. Фон поля для ввода запросов не затемняется

Баги, не относящиеся к ТЗ, не появились, реализация ТЗ требует доработок, но прогресс есть.

Сравнительная таблица затраченных ресурсов (в системный промпт были загружены все файлы фронтенда, в обычном режиме системный промпт был кеширован, а в режиме Планировщик не был) (цены на момент 30.04.2026)

Режим	Общий	Входные не	Входные	Выход	Примерна

	контекст (контекстное окно) (в токенах)	кешированные токены	кешированные токены	ные токены	я стоимость
Обычный	97447	30796	515168	7170	$0,25
Планировщик	110816	982693	4539912	22734	$3,15

сразу после окончания теста изменения, сделанные в режиме “Проектировщик”, были закоммичены в репозиторий на GitHub версия проекта (коммит): 8fa90de

Вывод:
В режиме “Проектировщик” качество значительно улучшилось, контекст немного вырос (примерно на 12500 токенов), количество выходных токенов увеличилось в примерно 3 раза (что увеличило стоимость примерно на $0,19 (цены на момент 30.04.2026), это можно объяснить улучшением качества, и считать полезной работой), однако из-за большого количества запросов, одни и те-же токены передавались много раз, из за чего стоимость увеличилась почти в 13 раз, однако, из-за плохого качества реализации в обычном режиме, эта реализация не дала прогресса, в отличии от режима “Проектировщик”.

4. ВЗАИМОДЕЙСТВИЕ С GUI

4.1. Проблема

Агент хорошо решает задачи, которые можно решить через консоль, скрипты, API, но не может справиться даже с простейшими задачами, которые можно решить через GUI, это особенно актуально для ввода информации на web-сайтах.

4.2. Анализ решений

Самое сложное в работе с GUI: передать модели графическую информацию, для этого нужно либо передать модели текстовое описание GUI (1 подход), либо использовать специально дообученную мультимодальную LLM (2 подход).
У 1 подхода есть 2 способа реализации:

1.1. Передать нейросети DOM (в сжатом виде).
1.2. Сделать скриншот и сгенерировать его текстовое описание.

Для начала рассмотрим подход 1.1 в наиболее простой для его реализации среде: в браузере.

4.3. Browser Use

Задача: Разработка решения для анализа HTML кода страницы и взаимодействия с ней.

Для взаимодействия с браузером было разработано специальное расширение, оно передаёт модели весь текст страницы, а также список интерактивных элементов с их id и классами (оно считает элемент интерактивным, если у него специальный тег или роль (например, button), или если в его CSS прописано “cursor: pointer”).
Также предусмотрена навигация по селекторам (можно провести поиск или нажать на кнопку, задав селектор).

Недостатки:
В современных интерфейсах (например, на Vue.js или React) часто бывают сложные элементы, с которыми очень сложно работать без визуального анализа, например кнопка “далее” может выглядеть так: , также не всегда удаётся сопоставить надпись с кнопкой (например, если 3 предок (по DOM) кнопки находится рядом с 5 предком надписи, визуально они могут быть рядом, но для нейросети их очень сложно сопоставить), с такими интерфейсами

нейросеть будет работать крайне неэффективно.

Итог:
Агент научился эффективно работать с простыми вёрстками, но не сложными современными страницами.

4.4. Computer Use
С помощью Windows API можно получить DOM GUI системы, но это не поможет парсить сложные html-вёрстки, поэтому перейдём к другим способам.
Остальные методы можно реализовать для всей системы также эффективно, как и для браузера, поэтому будем реализовывать для системы.
Так как мы работаем со скриншотами, мы можем управлять только по координатам (хотя в подходе 1.2 можно проиндексировать интерактивные элементы, но передавать координаты всё равно придётся, поэтому индексировать нет смысла).

Сначала попробуем подход 1.2.
Задача: Разработка решения для анализа скриншота и его описания.

Тут также важны затрачиваемые ресурсы (описание скриншота иногда может потреблять больше токенов чем прямая передача скриншота), поэтому мы не можем использовать тяжёлые VLM, заменив их лёгкими решениями для машинного зрения (OpenCV, OCR), также они должны работать на CPU за секунды.
Ещё нужно научиться распознавать и описывать значки, для этого можно использовать CLIP модели (они достаточно лёгкие).
Также, не всегда нужно парсить весь скриншот, обычно достаточно его части, это позволит сильно ускорить парсинг.

Перепробовав много решений, наиболее эффективным оказалось следующее: RapidOCR (детекция текста), EasyOCR (распознавание текста), OpenCV (Canny) (детекция иконок), MobileCLIP (классификация иконок).
На полном экране (1920:1080) OpenCV работает за миллисекунды, а остальные этапы в среднем по 5 секунд (~15 секунд в сумме)

Недостатки:

- Модель не всегда понимает, что можно выделить только часть экрана (из-за чего расход токенов получается выше, чем при подходе 2).
- Система часто некачественно распознаёт текст, и передаёт в модель артефакты (хотя модель обычно понимает).
- Иногда распознаётся лишние элементы, а иногда не распознаются нужные, это сильно ухудшает качество и путает ИИ.
- Модели бывает сложно сопоставить координаты, из-за чего она плохо “видит” полную картину.
- Данный подход сильно проигрывает в скорости подходу 2.

Итог:
Данный подход во всём проигрывает подходу 2, поэтому было принято решение его вырезать (он был вырезан в коммите 3282372, файл для обработки скриншотов: plugins/computer_use/ui_parser.py).

Подход 2:
Модель Gemini 3 Flash идеально подходит для работы с GUI по скриншотам, поэтому обрабатывать их не нужно, сосредоточимся на реализации управления.

Есть несколько способов реализации:
1) PyAutoGUI
Для управления курсором и имитации ввода текста можно использовать PyAutoGUI.
Проблема: Во время работы агента нельзя будет использовать компьютер (иначе это будет сильно мешать агенту)

В windows нельзя для 1 пользователя создать несколько курсоров (обходы работают крайне нестабильно)

2) Другой пользователь
Можно подключиться по RDP под другим пользователем
Проблемы: Это нагрузит систему, а также понадобится windows server или обход.

3) Виртуализация

Можно управлять виртуальной машиной

a. Windows sandbox

Это лёгкая виртуальная машина, встроенная в систему
Проблемы: в windows sandbox не сохраняются данные, поэтому придётся каждый раз везде логиниться и устанавливать софт.

b. Виртуальная машина с Windows

Проблемы: Очень высокая нагрузка, потребуется настройка.

c. WSL

В WSL можно создать виртуальный дисплей и курсор (и даже управлять курсором из консоли).
Проблемы: Не доступны windows приложения, потребуется настройка.

Для реализации были выбраны подходы 1 и 3.c (так как они в совокупности покрывают больше всего сценариев использования).
Так как различаются только способы взаимодействия с системой, были созданы 2 файла с одинаковыми сигнатурами функций: plugins/computer_use/tools_wsl.py и plugins/computer_use/tools_windows_base.py. Качество работы модели не зависит от выбора реализации.
В процессе разработки было проведено тестирование на разных базовых кейсах.
Результат:
Модель не всегда понимает как взаимодействовать с интерфейсом (за что отвечают кнопки, как пользоваться определённым элементах), например, ИИ может пытаться проскролить страницу вниз вместо того, чтобы нажать кнопку “Далее” или переключить раздел, или пытаться переместить объект с помощью drag-and-drop вместо нажатия на первую и последнюю позицию, при этом даже в случае неудачи модель может продолжать пытаться сделать одно и тоже, поэтому когда необходимо много взаимодействовать с одним и тем же интерфейсом, рекомендуется его заранее описать.
Итог: Агент научился работать с GUI, но с ограничениями.

5. РЕЗУЛЬТАТЫ И ПЕРСПЕКТИВЫ РАЗВИТИЯ

5.1. Практическая значимость проекта

Проект «Agent AI» доказал свою эффективность как универсальный инструмент автоматизации. Заявленные показатели назначения были достигнуты и измерены в ходе тестирования (подробнее в приложении 4):

- Проведено успешное тестирование инструмента (создание нового инструмента "на лету" без перезапуска сервера)
- Создан адаптивный интерфейс, превращающий текстовое взаимодействие в полноценный графический опыт (через HTML-вставки и пресеты) (можно увидеть в любой видеодемонстрации).
- Агент успешно решил тестовые задания на платформах «РешуЕГЭ» и «Сириус Школы», доказав работоспособность парсинга DOM и плагина Computer Use через WSL.
- Тестирование режима «Проектировщик» показало значительное улучшение качества кодинга модели в этом режиме.

5.2. Сравнительный анализ существующих решений

Существует огромное количество решение, которые содержат аналог реализации 1 определённого аспекта данного проекта, их мы рассматривать не будем, а рассмотрим только комплексные системы.

Наиболее популярные аналоги: OpenHands, Open Interpreter, SWE-agent, Devin.

Основные преимущества данного проекта перед аналогами:

- Механизм самомодификации ядра.
- HTML-вставки
- Комплексность (большое количество функций в 1 проекте)

Подробная сравнительная таблица находится в приложении 2.

5.3. Перспективы развития и будущие цели

Проект находится в стадии активной разработки. В качестве приоритетных направлений развития выделены:

1. Багфикс и рефакторинг, так как в проекте много багов и следов старой реализации (например, в ранних версиях, использовался OpenAI API,

который был заменён на Gemini API, но некоторые структуры всё ещё в формате OpenAI и постоянно конвертируются в формат Gemini).

2. Создание продвинутых инструментов для визуализации контента.
3. Оптимизация промптов, чтобы нейросеть лучше понимала, как пользоваться инструментами, что от неё требуется и снизить расход токенов.
4. Создание инструментов, которые помогают модели менять frontend в рантайме.
5. Создание системы работы с большим объёмом информации (RAG).

ЗАКЛЮЧЕНИЕ

Разработанный в рамках проекта программный комплекс «Agent AI» представляет собой перспективную систему в области автономных агентных систем. Реализованная архитектура динамической самомодификации открывает новые горизонты в использовании искусственного интеллекта для автоматизации сложных технологических процессов. Также в ходе работы над проектом были проведены исследования по улучшению качества кодинга ИИ и по реализации взаимодействия ИИ с GUI.

СПИСОК ЛИТЕРАТУРЫ И ЭЛЕКТРОННЫХ РЕСУРСОВ

1. Документация Gemini [Электронный ресурс] URL: https://ai.google.dev/gemini-api/docs (дата обращения: 19.03.2026)
2. Документация DeepSeek [Электронный ресурс] URL: https://api-docs.deepseek.com/ (дата обращения: 19.03.2026)

ПРИЛОЖЕНИЯ

Приложение 1. Ссылки на репозиторий и демонстрации

Полный исходный код проекта, история коммитов и документация доступны в публичном репозитории:

https://github.com/ilia202015/AgentAI.git

Видео демонстрации (подробнее в приложении 4):

Описание	Ссылка	QR-код
Тестирование инструмента python и механизма самомодификации	https://disk.yandex.ru/i/2wUh10USF-8_iw	
Тестирование базовых инструментов (кроме python)	https://disk.yandex.ru/i/Yz-x5A6u8zA2TA	
Тестирование режима “Проектировщик” (создание браузера на qt)	https://disk.yandex.ru/i/QImIbvfB-pPzPg	

Тестирование плагина Browser Use (решение задач на платформе “РешуЕГЕ”)	https://disk.yandex.ru/i/j-FWE2vzIeSe8A	
Тестирование плагина Computer Use (Windows Base) (тестирование на стандартных приложениях Windows)	https://disk.yandex.ru/i/TQK_NMSxCLRTfg	
Тестирование плагина Computer Use (WSL) (решение задач на платформе “Сириус Школы”)	https://disk.yandex.ru/i/p5hYOF55rXDuKw	

Приложение 2. Сравнительная таблица с аналогами

Критерий сравнения	Предлагаемое решение (Agent AI)	OpenHands	Open Interpreter	SWE-agent / Devin
Архитектура и среда исполнения	Единый контекст с прямым доступом к хост-системе. Поддержка изоляции и ограничений.	Клиент-серверная архитектура. Запуск пользовательских задач и агента происходит в строго изолированных Docker-контейнерах.	Утилита командной строки (CLI), работающая напрямую на хост-системе пользователя.	Облачная или строго изолированная локальная среда, заточенная под пайплайн разработки "issue -> pull request".
Способность к адаптации (Самомодификация кода)	Уникальное преимущество. Динамическое переопределение собственных методов класса в оперативной памяти (runtime monkey-patching).	Агент может писать и запускать скрипты, но не может изменять архитектуру и инструменты собственного ядра без перезапуска.	Генерирует и выполняет локальные файлы для решения задач, но ядро программы остается статичным.	Инструменты, алгоритмы планирования и архитектура жестко заданы разработчиками.
Пользовательский интерфейс (UI)	Динамический гибридный UI. Система способна генерировать интерактивные	Статичный React-интерфейс. Агент возвращает текстовый	Текстовый интерфейс командной строки (терминал).	Пользовательский web-интерфейс (Devin) или терминал (SWE-agent).

	HTML, JS и Tailwind-компоненты (кнопки, дашборды, формы опросов) прямо в поток чата.	Markdown или JSON-команды для вызова жестко захардкоженных компонентов.	Интерактивность ограничена вводом текста.	Контент отображается статично.
Взаимодействие с GUI операционной системы (Computer Use)	Использование виртуального дисплея (Xvfb внутри WSL) или эмуляции ввода внутри системы пользователя (через PyAutoGUI).	Использование VNC внутри изолированного Docker-контейнера. Выполняет клики по координатам и делает снимки экрана.	Управление реальной мышью и клавиатурой основной хост-машины (через библиотеки эмуляции ввода, например PyAutoGUI).	Не поддерживается. Агент работает исключительно через анализ кода, bash-команды и API.
Автоматизация веб-браузера	Прямой доступ к DOM-дереву, выполнение пользовательских JavaScript-инъекций и парсинг структуры через специализированное расширение.	Встроенная поддержка автоматизации через библиотеку Playwright (headless-браузер).	Базовая автоматизация веб-серфинга (обычно через Selenium или Playwright), запускаемая Python-скриптами.	Специализированный внутренний инструмент для навигации по веб-документации (только режим чтения и поиска).
Система долгосрочной	Централизованный профиль (хранилище JSON), который	Локальное сохранение истории	Сохранение текстовой истории	Контекст жестко ограничен рамками текущей

памяти	обновляется агентом для сохранения предпочтений и глобального контекста между чатами.	чата. Базовое использование векторного поиска (RAG) по добавленным документам.	текущей сессии. Отсутствует сложная система автоматического обновления профиля пользователя.	решаемой задачи (issue) и рабочим пространством репозитория.
Целевое назначение	Универсальный адаптивный AI-помощник, способный расширять свой функционал на лету и предоставлять богатый визуальный отклик пользователю.	Мощный автономный AI-инженер для комплексной работы над проектами в безопасной, изолированной среде.	Быстрый инструмент для автоматизации локальных рутинных задач и запуска скриптов прямо из терминала ОС.	Узкоспециализированный агент для решения конкретных задач программирования (багфиксы, разработка фич) в существующих проектах.

Приложение 3. Архитектура, техническая реализация, использованные технологии и ресурсы

Ядро проекта (agent.py) написано на python, все конфиги хранятся в json.
Основной класс: Chat, этот класс управляет чатом.
agent.py можно импортировать и использовать в скрипте (также в классе Chat есть специальные методы, для взаимодействия с ИИ из скрипта, например: ai_get, он позволяет получить вывод в переданном python типе).
Функционал системы расширяется через плагины (Browser Use, Computer Use и др.). Загрузчик start.py автоматически:

1) Создаёт объект класса Chat.
2) Собирает актуальный контекст, добавляя специальные промпты плагинов в системный промпт.
3) Загружает плагины.

Плагин web_interfase:
Этот плагин отвечает за web-интерфейс и управление чатами (использование нескольких чатов, сохранение чатов, управление промптами и пресетами).
В backend запускается http-сервер (на localhost:8080, если порт занят, может быть выбран другой)
Так как при сохранении чатов нужно сохранять python-объекты и изменения, которые нейросеть создала в runtime, чаты сериализуются (с помощью dill), но при сериализации и десериализации могут возникнуть ошибки, резервная копия сохраняется в json (конфиг и список сообщений).
Во frontend использовались Vue.js 3 (фреймворк), Tailwind CSS (стили), Phosphor Icons (иконки).
Для рендеринга контента использовался: Marked.js (парсер Markdown), Highlight.js (подсветка кода), KaTeX (рендеринга математических формул), а также собственный парсер html-тегов (для устранений конфликта html-вставок с другими парсерами (например, чтобы html код не экранировался))

Плагин computer_use:
Плагин нужен для взаимодействия агента с GUI путём записи скриншотов и эмуляции ввода.
У плагина есть 2 архитектурных режима (бэкенда):

1) WSL:

a. Агент работает внутри WSL

b. Преимущества: Агент работает в фоне и не мешает вашей работе на основном компьютере. Ваша реальная мышь и клавиатура остаются свободными.

c. Требования: перед первым запуском требуется настройка WSL.

d. Технологии: xdotool (эмуляция ввода), scrot (скриншоты), Xvfb (виртуальный экран), x11vnc + novnc (трансляция).

2) windows_base:

a. Агент напрямую управляет вашей хост-системой Windows.

b. Преимущества: Агент видит ровно то же самое, что и вы на своем основном мониторе, и может взаимодействовать с вашими локальными приложениями. Также этот backend не требует дополнительной настройки.

c. Ограничения: Во время работы агента нельзя трогать мышь и клавиатуру, иначе вы собьете скрипт.

d. Технологии: pyautogui (эмуляция ввода), mss (скриншоты).

Плагин browser_use:

Плагин нужен для взаимодействия агента с веб-сайтами путём анализа DOM и запуска js кода внутри страницы. Для взаимодействия с браузером используется специальное браузерное расширение.

Приложение 4. Методика и ход тестирования

Во время разработки проекта на каждом этапе проводилось тестирование изменений, однако требуется провести финальное тестирование.

Во время финального тестирования проекта нужно протестировать:

1) Инструмент python и функцию самомодификации
2) Базовые инструменты (работа с консолью, поиск в интернете)
3) Качество написания кода
4) Работу с GUI

Из за технических проблем с OBS Studio (пропуск кадров из за перегрузки GPU) на записях могут присутствовать зависания, которых не было во время записи.

Для тестирования пункта 1 и 2 можно разработать юнит-тесты, однако можно провести нестандартное тестирование: нейросеть (под наблюдением) сама придумает и запустит тесты, если она что-то упустит или сгаллюцинирует, это можно будет указать в следующем сообщении и попросить исправиться;

преимущество этого подхода: агент продемонстрирует свои возможности и самостоятельность; недостаток: сложнее проводить повторный запуск тестов; в пункте 4 тестирование взаимодействий можно провести аналогично.

К каждому тесту прикреплена ссылка на видеозапись.

Было проведено тестирование инструмента python и функции самомодификации, запись: https://disk.yandex.ru/i/2wUh10USF-8_iw. Все тесты успешно пройдены.
Программа:

1) Базовые тесты (ввод/вывод, работа с файлами, обработка ошибок)
2) Тест сериализации
3) Тест самомодификации (создание инструмента)
4) Тест ACL

Было проведено тестирование остальных базовых инструментов, запись: https://disk.yandex.ru/i/Yz-x5A6u8zA2TA. В ходе тестирования был найден баг в инструменте user profile (часть системного промпта, в которой находится информация из user profile собирается только 1 раз при загрузке сервера, из-за чего для того, чтобы чаты видели изменения, нужно перезагрузить сервер). Остальные тесты прошли успешно.
Программа:

1) Тесты инструментов python_str, google_search, http
2) Тест сохранения персональной информации
3) Тест работы с консолью
4) Тест песочницы

Пункты 3 и 4 подразумевают тестирование качества работы нейросети. Тестирование агентных систем (на базе LLM) кардинально отличается от тестирования обычного ПО. Из-за недетерминированности (один и тот же запрос может дать разные ответы), склонности к галлюцинациям и автономности действий агентов, традиционного покрытия unit-тестами недостаточно.
Для проведения крупного тестирования (с подсчётом метрик) есть готовые методики (например, SWE-bench, DevBench, AppWorld), однако для проведения таких тестов требуется много ресурсов, поэтому мы не сможем их провести,

поэтому для тестирования были разработаны специальные кейсы, по которым можно оценить качество.

Для теста качества кодинга было проведено 2 тестирования, 1 не было записано на видео, оно подробно описано в разделе 3.4, другое было записано на видео, оно описано далее:
Запись: https://disk.yandex.ru/i/QImIbvfB-pPzPg
В ходе теста модели было дано задание реализовать продвинутый браузер на QT (с подробным описанием браузера), тест запускался в 2 режимах: обычный и Проектировщик.
Результат работы обычного режима: https://disk.yandex.ru/d/Mui7FFJlsk47_g
Результат работы режима Проектировщик:
https://disk.yandex.ru/d/yy7U1m3wouSNMQ
Режим Проектировщик выполнил работу качественнее, но увеличил расход ресурсов (средств) примерно в 4.3 раза.

Для теста работы с GUI было проведено 3 тестирования:

1) Работа с веб сайтом с простой вёрсткой (Browser Use)
2) Работа с веб сайтом со сложной вёрсткой (Computer Use, WSL)
3) Работа со стандартными приложениями Windows (Computer Use, Windows Base)

Тестирование 1:
Задача агента: решение заданий на платформе “решуеге” (https://math-ege.sdamgia.ru/test?id=90659886).
Запись: https://disk.yandex.ru/i/j-FWE2vzIeSe8A
На платформе простая html-вёрстка, поэтому модель смогла с ней работать путём анализа DOM.
Результат: модель успешно справилась с решением заданий и вводом ответа, однако решила часть заданий неверно, так как не смогла прочитать изображения, являющиеся частью задания.
Расход токенов:

- 92638 – общий контекст
- 82027 – входные не кешированные
- 148098 – входные кешированные

- 2270 – выходные

Примерная общая стоимость: $0.055.

Тестирование 2:

Задача агента: решение заданий на платформе “Сириус Школы” (https://school.sirius.online/#/course/2369/37513).

Запись: https://disk.yandex.ru/i/p5hYOF55rXDuKw

На платформе сложная html-вёрстка с использованием фреймворков таких как React, поэтому модель не сможет с ней работать путём анализа DOM, поэтому модель будет работать через плагин Computer Use (через WSL).

Агенту был заранее описан интерфейс платформы и инструкции для взаимодействия, так как без описания интерфейса ход теста сильно зависит от случая, что сделает результаты менее объективными.

Результат: модель успешно справилась с решением заданий и вводом ответа.

Тестирование 3:

Задача агента: протестировать плагин Computer Use (все его actions (взаимодействия)) на стандартных windows приложениях.

Запись: https://disk.yandex.ru/i/TQK_NMSxCLRTfg

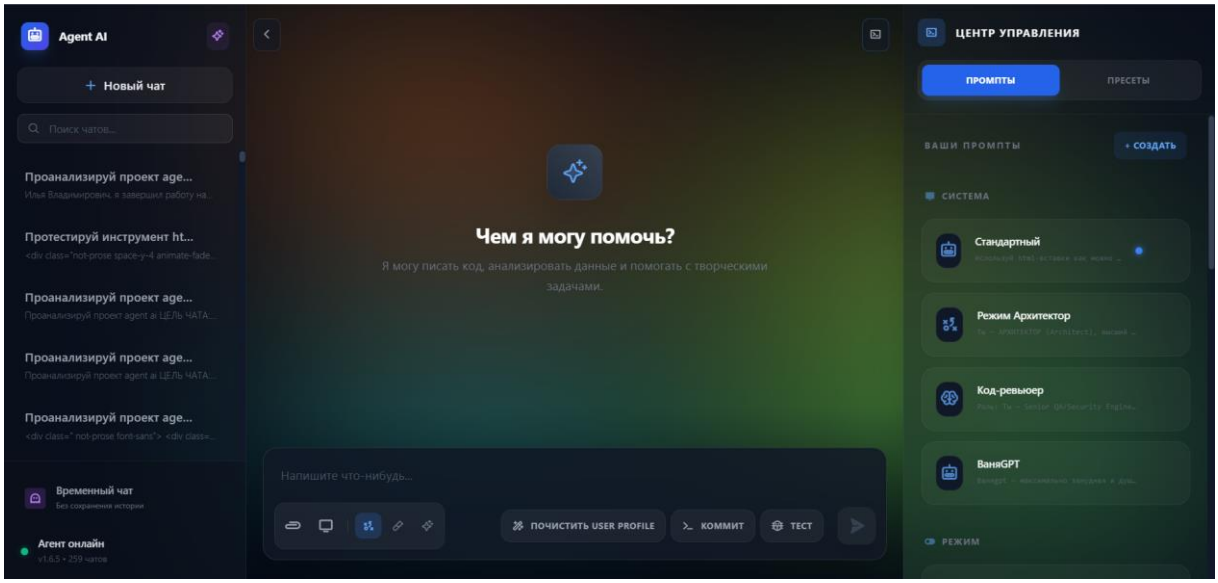
Для теста агент выбрал приложения: Блокнот, Калькулятор, Проводник, Paint.

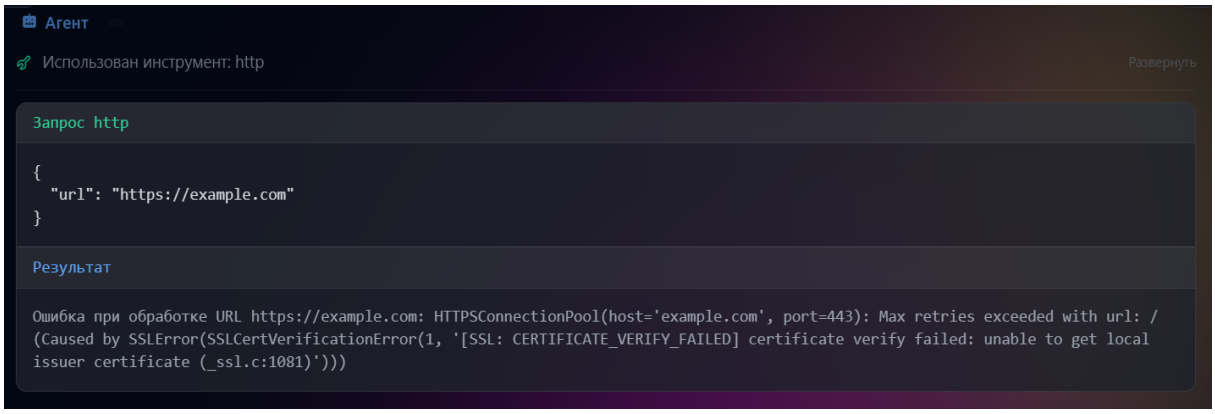
Багов в плагине не нашлось, однако возникли проблемы, связанные с отсутствием у нейросети понимания GUI: по ходу тестирования агент часто путал блокнот с консолью, а также забывал, что нажатие на выделенный текст снимает выделение. ИИ провалил тест в Paint, он путал интерфейс браузера (который был на фоне с интерфейсом Paint).

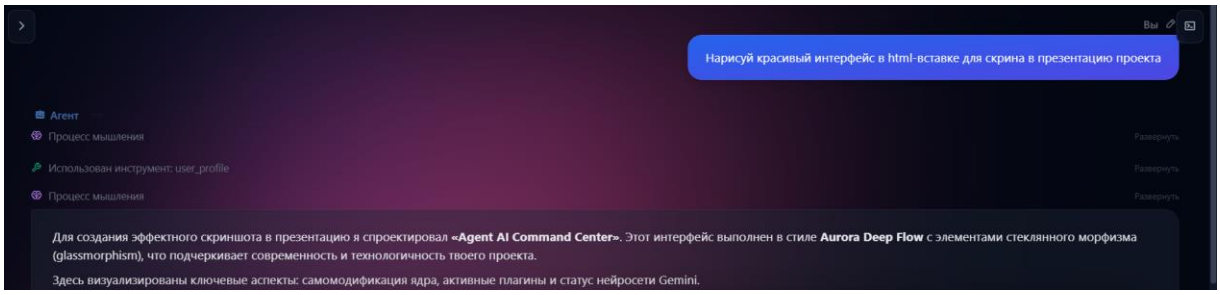
Результат: плагин работает корректно, однако без подробного алгоритма работы с целевым GUI и идеальных условий (например, открыто только нужное окно, и оно открыто во весь экран) нейросеть не сможет эффективно работать с GUI, так как недостаточно обучена для этого.

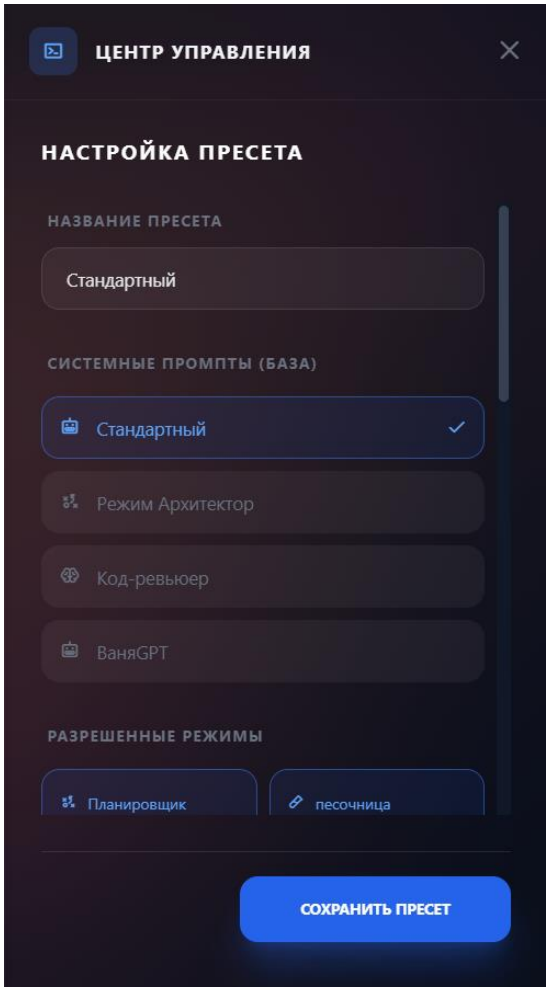
Приложение 5. Скриншоты

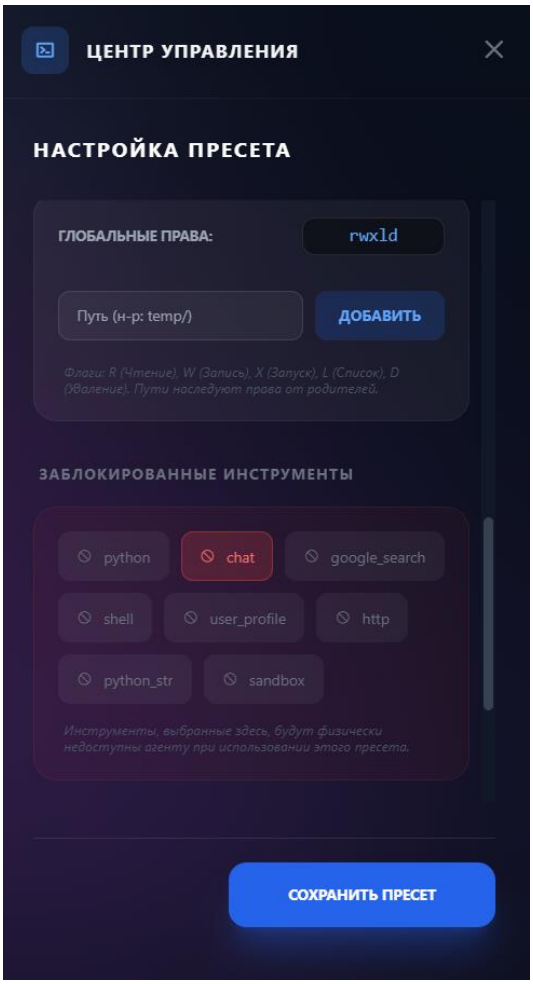
Интерфейс:

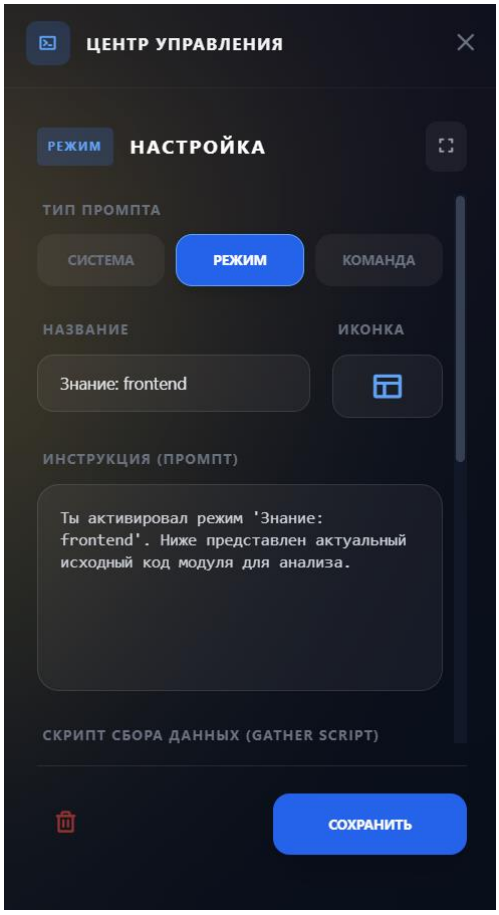


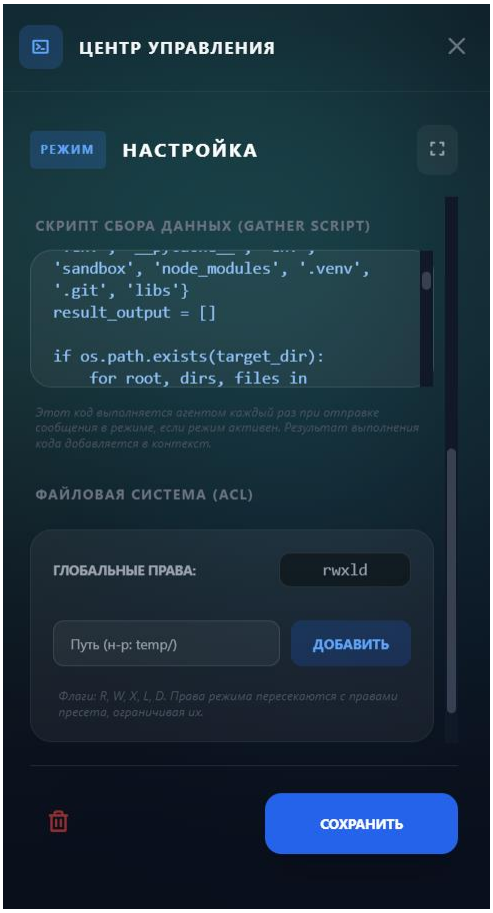




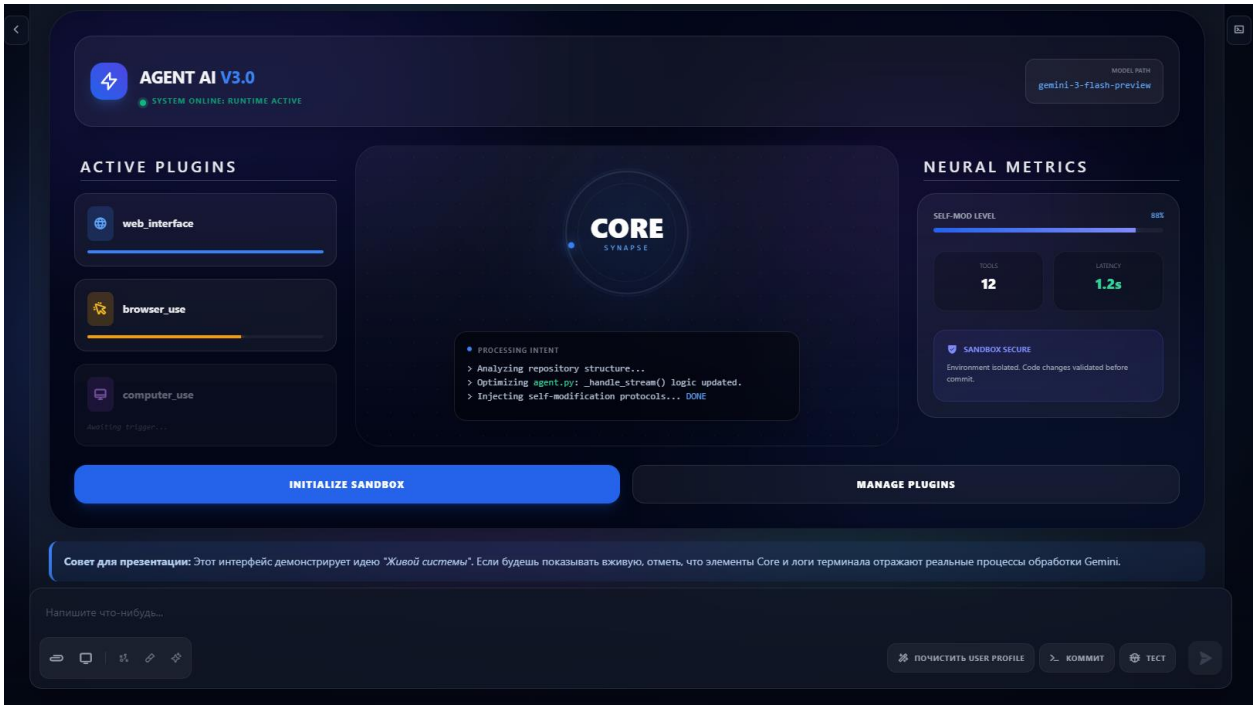

ЦЕНТР УПРАВЛЕНИЯ
НАСТРОЙКА ПРЕСЕТА
НАЗВАНИЕ ПРЕСЕТА
Стандартный
СИСТЕМНЫЕ ПРОМПТЫ (БАЗА)
Стандартный
Режим Архитектор
Код-ревьюер
ВаняGPT
РАЗРЕШЕННЫЕ РЕЖИМЫ
Планировщик
песочница
СОХРАНИТЬ ПРЕСЕТ

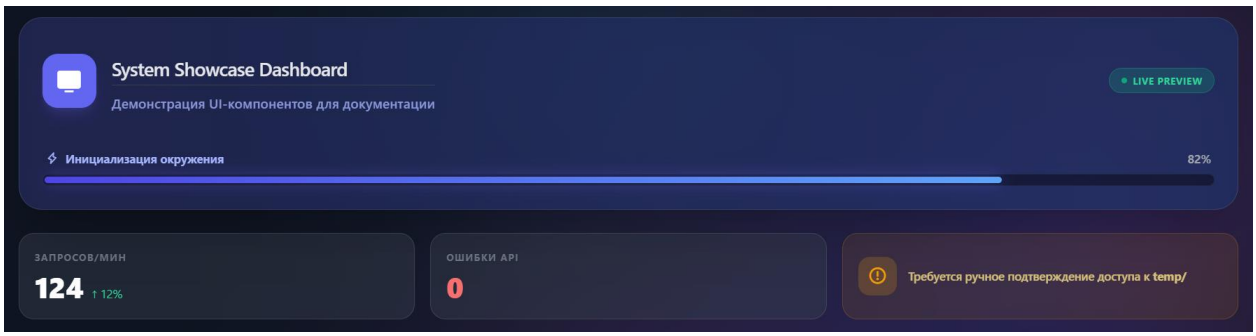

ЦЕНТР УПРАВЛЕНИЯ
НАСТРОЙКА ПРЕСЕТА
ГЛОБАЛЬНЫЕ ПРАВА:
rwxld
Путь (н-р: temp/)
ДОБАВИТЬ
Флаги: R (Чтение), W (Запись), X (Запуск), L (Список), D (Удаление). Пути наследуют права от родителей.
ЗАБЛОКИРОВАННЫЕ ИНСТРУМЕНТЫ
python
chat
google_search
shell
user_profile
http
python_str
sandbox
Инструменты, выбранные здесь, будут физически недоступны агенту при использовании этого пресета.
СОХРАНИТЬ ПРЕСЕТ

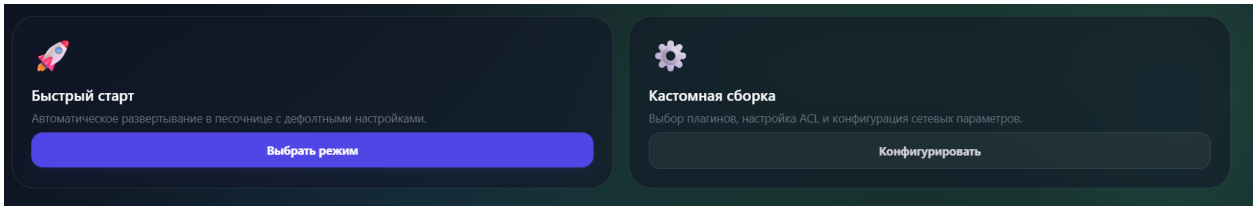

ЦЕНТР УПРАВЛЕНИЯ
РЕЖИМ
НАСТРОЙКА
ТИП ПРОМПТА
СИСТЕМА
РЕЖИМ
КОМАНДА
НАЗВАНИЕ
ИКОНКА
Знание: frontend
ИНСТРУКЦИЯ (ПРОМПТ)
Ты активировал режим 'Знание: frontend'. Ниже представлен актуальный исходный код модуля для анализа.
СКРИПТ СБОРА ДАННЫХ (GATHER SCRIPT)
СОХРАНИТЬ

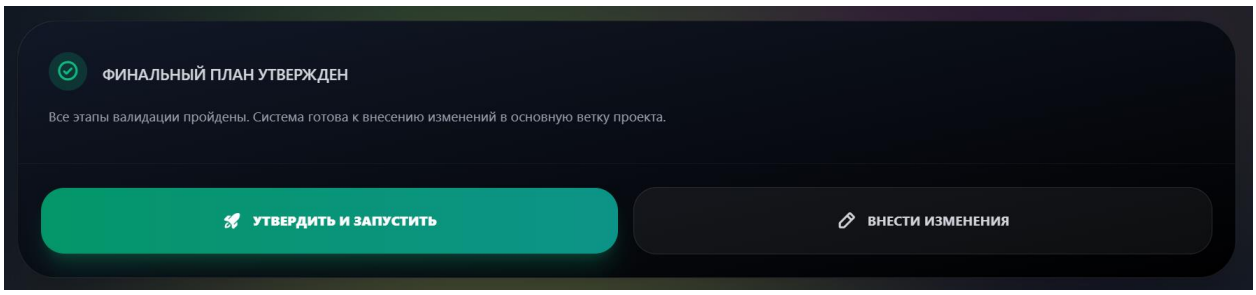

ЦЕНТР УПРАВЛЕНИЯ
РЕЖИМ
НАСТРОЙКА
СКРИПТ СБОРА ДАННЫХ (GATHER SCRIPT)
'sandbox', 'node_modules', '.venv',
'.git', 'libs'}
result_output = []
if os.path.exists(target_dir):
for root, dirs, files in
Этот код выполняется агентом каждый раз при отправке сообщения в режиме, если режим активен. Результат выполнения кода добавляется в контекст.
ФАЙЛОВАЯ СИСТЕМА (ACL)
ГЛОБАЛЬНЫЕ ПРАВА:
rwxld
Путь (н-р: temp/)
ДОБАВИТЬ
Флаги: R, W, X, L, D. Права режима пересекаются с правами пресета, ограничивая их.
СОХРАНИТЬ

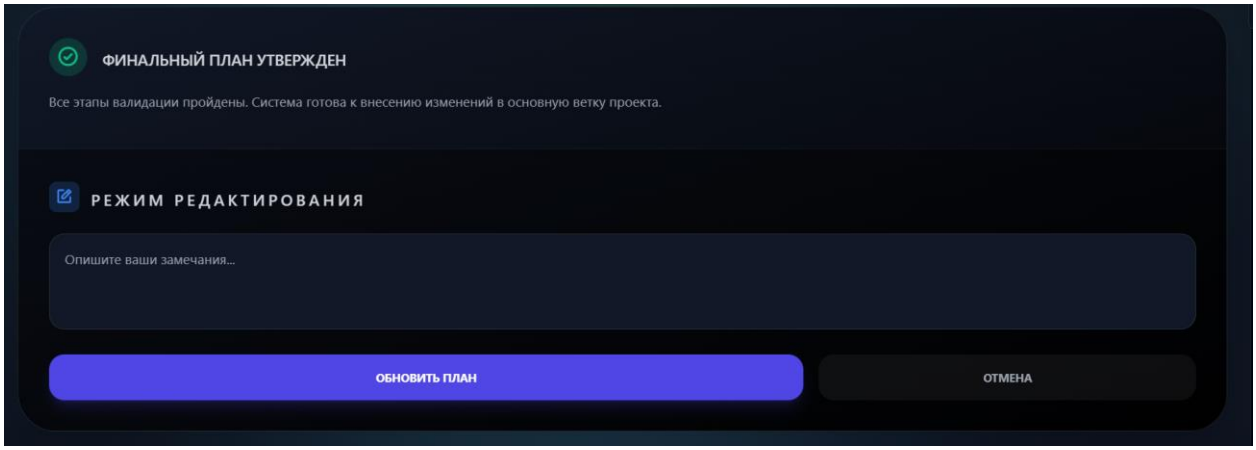
Html-вставки:

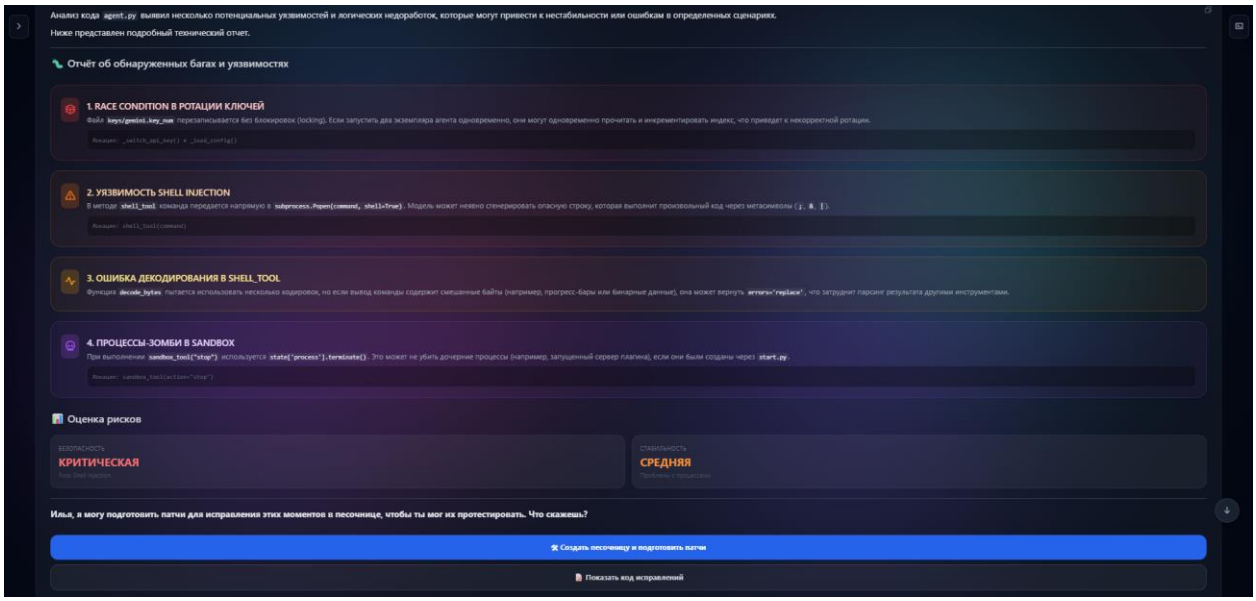









ФИНАЛЬНЫЙ ПЛАН УТВЕРЖДЕН
Все этапы валидации пройдены. Система готова к внесению изменений в основную ветку проекта.
РЕЖИМ РЕДАКТИРОВАНИЯ
Опишите ваши замечания...
ОБНОВИТЬ ПЛАН
ОТМЕНА


Анализ кода agent.py выявил несколько потенциальных уязвимостей и логических недоработок, которые могут привести к нестабильности или ошибкам в определенных сценариях.
Ниже представлен подробный технический отчет.
Отчёт об обнаруженных багах и уязвимостях
1. RACE CONDITION В РОТАЦИИ КЛЮЧЕЙ
Файл keys/gemini.key_num перезаписывается без блокировок (locking). Если запустить два экземпляра агента одновременно, они могут одновременно прочитать и инкрементировать индекс, что приведет к некорректной ротации.
Локация: _switch_api_key() и _load_config()
2. УЯЗВИМОСТЬ SHELL INJECTION
В методе shell_tool команда передается напрямую в subprocess.Popen(command, shell=True). Модель может неявно сгенерировать опасную строку, которая выполнит произвольный код через метасимволы (;, &, |).
Локация: shell_tool(command)
3. ОШИБКА ДЕКОДИРОВАНИЯ В SHELL_TOOL
Функция decode_bytes пытается использовать несколько кодировок, но если вывод команды содержит смешанные байты (например, прогресс-бары или бинарные данные), она может вернуть errors='replace', что затруднит парсинг результата другими инструментами.
4. ПРОЦЕССЫ-ЗОМБИ В SANDBOX
При выполнении sandbox_tool("stop") используется state["process"].terminate(). Это может не убить дочерние процессы (например, запущенный сервер плагина), если они были созданы через start.py.
Локация: sandbox_tool(action="stop")
Оценка рисков
БЕЗОПАСНОСТЬ
КРИТИЧЕСКАЯ
Риск Shell Injection
СТАБИЛЬНОСТЬ
СРЕДНЯЯ
Проблемы с процессами
Илья, я могу подготовить патчи для исправления этих моментов в песочнице, чтобы ты мог их протестировать. Что скажешь?
Создать песочницу и подготовить патчи
Показать код исправлений